Software Architecture and Design Specification (SAD)

Project: Healthcare Appointment and Patient Record Manager

Version: 1.0 (initial draft)

Authors:

KARANAM SREEHITHA – PES1UG24AM129

KHUSHI PATEL – PES1UG24AM134

KIRTHANA S -- PES1UG24AM136

PORLA NEHA -- PES1UG25AM808

Status: Reviewed by Project Team

Revision History

Version	Date	Author	Change Summary	Approval
1.0	28-10-2026	Project team	Initial project-specific draft adapted from provided template	Approved by Project Team

Approvals

Role	Name	Signature / Email	Date
Team Lead	KARANAM SREEHITHA	sreehithakaranam@gamil.com	28-10-2026
Team Member	KHUSHI HIRENKUMAR PATEL KIRTHANA S PORLA NEHA	khushihpatel06@gmail.com Kirthanashetty17@gmail.com porlaneha1@gmail.com	28-10-2026
Course TA / Evaluator			

1. Introduction

1.1 Purpose

This document describes the current architecture and design of the Healthcare Appointment and Patient Record Manager and identifies planned components separately from verified implementation.

1.2 Scope

Current verified scope: Flask web routes and HTML templates for patient and appointment workflows backed by SQLite. Authentication, medical history, prescriptions and dashboard components are not verified in the current baseline.

1.3 Audience

Developers, testers, team lead, TA/instructor and maintainers.

1.4 Definitions

Flask: Python web framework; SQLite: embedded relational database; CRUD: Create, Read, Update, Delete; SAD: Software Architecture and Design; RTM: Requirements Traceability Matrix.

2. Document Overview

2.1 How to use this document

Use this document with the SRS and STP. Architecture diagrams and descriptions represent the verified baseline unless explicitly labeled planned.

2.2 Related Documents

SRS_Healthcare_Appointment_Patient_Record_Manager.docx; STP_Healthcare_Appointment_Patient_Record_Manager.docx; project README and source code.

3. Architecture

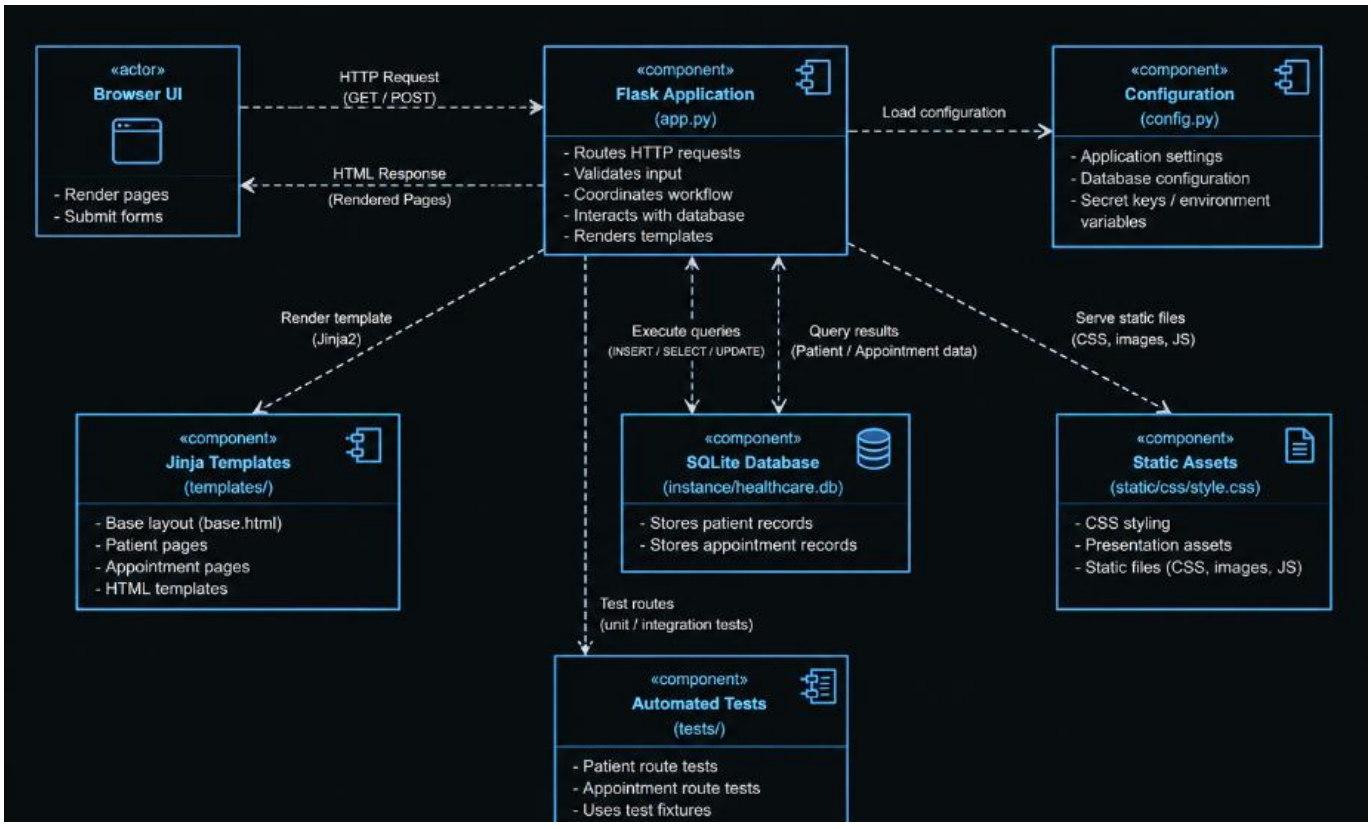
3.1 Goals & Constraints

- Provide simple and understandable patient and appointment workflows for a coursework prototype.
- Separate HTTP routing/business validation, persistence and presentation templates.
- Use SQLite for straightforward local development and testing.
- Constraints: local SQLite storage; no verified production deployment; no verified authentication/authorization layer; avoid real patient data.

3.2 Stakeholders & Concerns

Stakeholder	Primary concerns
Clinic staff / intended users	Correct records, simple navigation, useful validation feedback.
Developers	Clear modules, testability, maintainable code, predictable setup.
Testers	Repeatable test data, clear expected results, requirement traceability.
TA / evaluator	Evidence of requirements analysis, design, tests, sprint progress and CI.
Patients / data subjects	Privacy and correctness; prototype must not be treated as production-ready.

3.3 Component (UML) Diagram



3.4 Component Descriptions

Component	Responsibility	Current status
app.py	Defines app routes, database connection/initialization and patient/appointment validation and workflows.	Verified
config.py	Holds configuration values used by the application.	File present; detailed configuration to be reviewed
SQLite schema	Stores patients and appointments; schema initialization is in application code.	Verified tables
templates/base.html	Shared page structure/layout.	File present
templates/patients/*	Patient list/register, view and edit pages.	Files present
templates/appointments/*	Appointment list/book/edit pages.	Files present
static/css/style.css	Styling for the browser UI.	File present
tests/*	Pytest fixtures and tests for patient and appointment behavior.	31 tests passed in latest reported run
Authentication / history / prescriptions / dashboard	Potential future components.	Not verified

3.5 Chosen Architecture Pattern and Rationale

A lightweight layered web architecture is used: browser/presentation templates, Flask route and validation logic, and SQLite persistence. This is suitable for a small educational application because it minimizes setup and is straightforward to test. A distributed microservice architecture would add unnecessary operational complexity for the current scope.

3.6 Technology Stack & Data Stores

Layer	Technology	Purpose
Web application	Python / Flask	HTTP routing and request handling.
Presentation	Jinja HTML templates, CSS	Page rendering and styling.
Persistence	SQLite	Local relational storage for patient and appointment records.
Testing	pytest; coverage tooling where configured	Automated behavior checks and coverage measurement.
Version control / CI	GitHub; GitHub Actions planned/being configured	Collaborative source management and automated checks.

3.7 Risks & Mitigations

Risk	Mitigation
Unauthorized access to sensitive records	Do not use real patient data; implement and test authentication/authorization before any realistic deployment.
Database corruption or accidental deletion	Use test fixtures and backups for development; test delete behavior and related-record constraints.
Differences between developer environments	Document Python/dependency setup and run tests in CI.
Requirements and implementation drift	Maintain the RTM; update SRS/SAD/STP when sprint scope changes.
Single-file route logic becomes hard to maintain	Refactor into modules only when useful and keep regression tests.

3.8 Traceability to Requirements

SRS requirement group	Design component
FR-001 to FR-007 (patient workflows)	Patient routes, validation functions, patient templates, patients table.
FR-008 to FR-013 (appointment workflows)	Appointment routes, validation functions, appointment templates, appointments table.
FR-014 to FR-015 (health/error paths)	Health endpoint and controlled route responses.
NFR-002 / NFR-003 (integrity/validation)	SQLite persistence and server-side validation.
SR-001 to SR-005 (security)	Deployment/access-control/input-handling/repository practices; some requirements are planned or unverified.

3.9 Security Architecture

Current verified baseline does not establish production-grade authentication, authorization, HTTPS deployment or encryption at rest. Until those are implemented and verified, the application should be treated as a coursework prototype and populated only with synthetic records.

- Validate form inputs on the server before database writes.
- Use parameterized database operations rather than string-concatenated SQL.
- Do not commit secrets, real patient data, local database files or coverage artifacts unless explicitly intended and sanitized.
- Protect the repository and deployment configuration; use HTTPS for any deployed instance handling sensitive information.

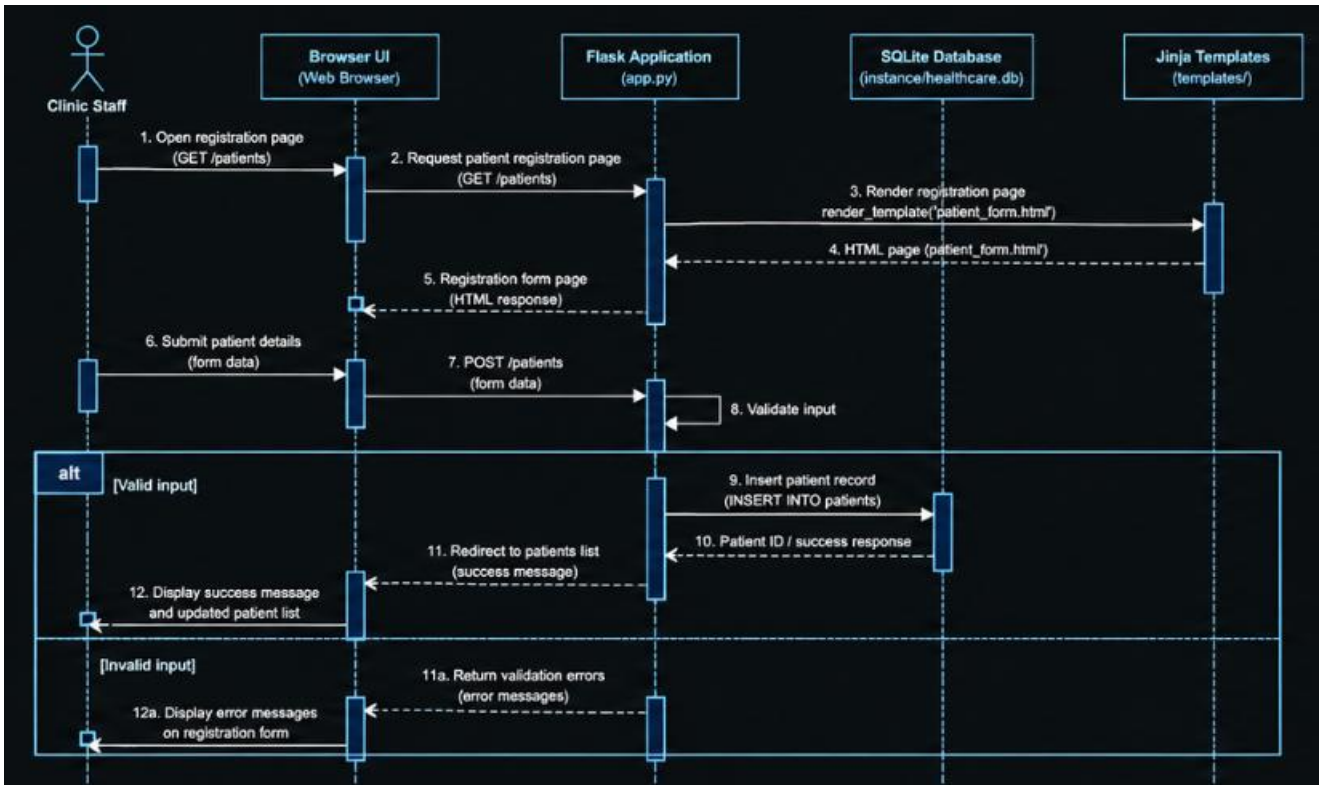
- Add role-based access and audit logging only as a planned change until verified.

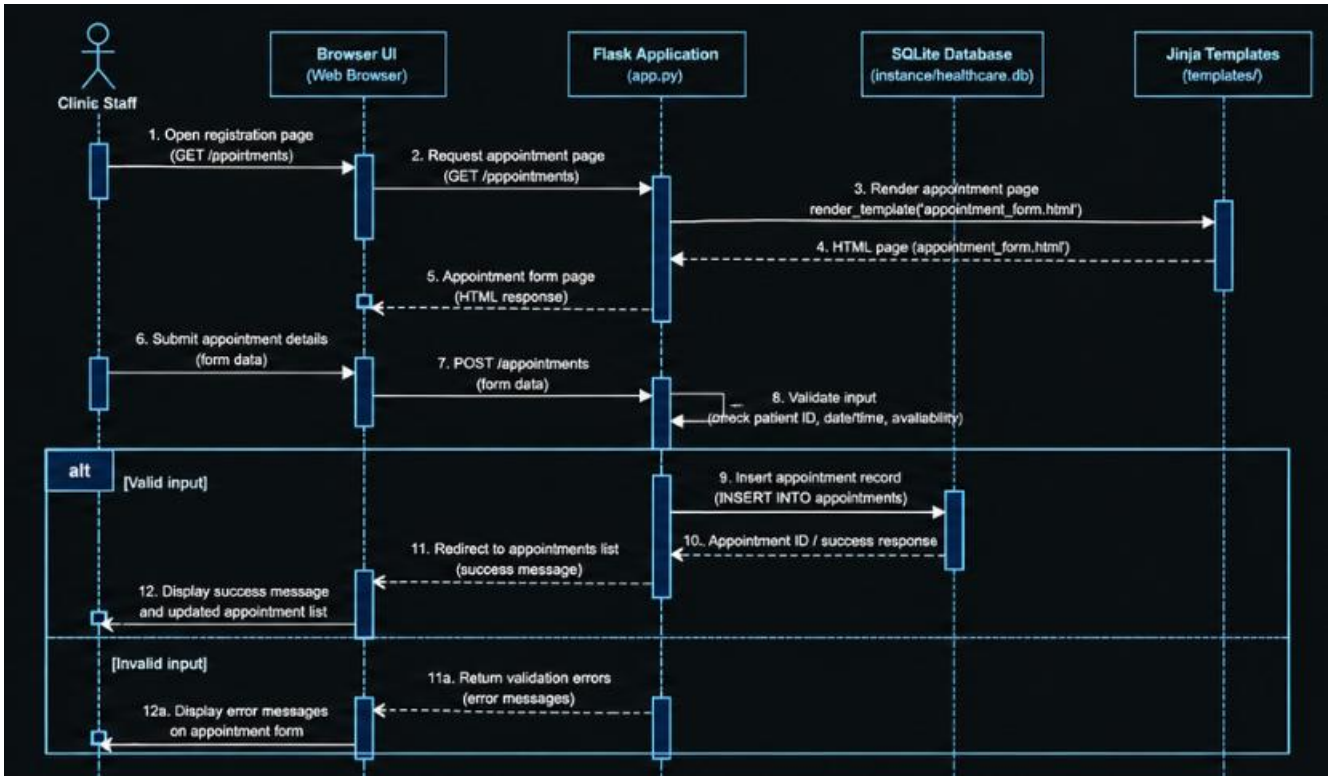
4. Design

4.1 Design Overview

A request enters a Flask route, is validated, reads or writes the SQLite database, and returns a rendered template or a controlled response. Patient and appointment workflows share the application and database but are separated by routes, templates and tables.

4.2 UML Sequence Diagrams





4.3 API Design

The current project is primarily a server-rendered web application; the routes below are page/form interfaces, not necessarily a REST JSON API. Confirm exact HTTP methods and parameter names against app.py before final approval.

Route / interface	Purpose	Inputs	Expected behavior
GET /	Render landing/home page.	None	Returns the home page.
GET /health	Basic health check.	None	Returns the configured health response.
GET/POST /patients	List/search patients and submit patient registration.	Form fields / search filters	Valid submissions persist; invalid input is rejected.
GET /patients/<id>	View patient details.	Patient ID in path	Displays selected record or controlled missing-record response.
GET/POST /patients/<id>/edit	Edit a patient.	Patient ID and updated fields	Valid updates persist; invalid input is rejected.
POST /patients/<id>/delete (verify method)	Delete a patient.	Patient ID	Deletes according to configured relationship rules.
GET/POST /appointments	List/filter appointments and submit booking.	Appointment fields / filters	Valid booking persists; invalid input is rejected.
GET/POST /appointments/<id>/edit	Edit appointment details.	Appointment ID and fields	Valid updates persist.
POST /appointments/<id>/status (verify method)	Update appointment status.	Appointment ID and status	Supported status change persists.

4.4 Error Handling, Logging & Monitoring

Validation errors should be returned in a controlled way and should not expose stack traces or database internals to end users. Developer logs should omit sensitive data. CI should run the test suite on pushes and pull requests; failed checks should block approval according to team policy.

4.5 UX Design

- Use consistent navigation and page titles.
- Label required fields and show validation feedback near the relevant form.
- Use clear empty states when no patients or appointments match a query.
- Use synthetic data during demonstrations and avoid displaying unnecessary personal details.

4.6 Open Issues & Next Steps

- Verify exact routes, HTTP methods, database schema and foreign-key/delete behavior against the latest source.
- Confirm whether authentication, medical history, prescription management and dashboard are in the sprint plan; document them as planned until implemented.

- Add/verify GitHub Actions workflow and capture a successful CI run.
- Render UML diagrams and ensure diagram content matches the implementation.
- Update this document after material design or scope changes.

5. Appendices

5.1 Glossary

Flask, SQLite, CRUD, FR, NFR, SRS, SAD, STP, RTM, CI/CD, UML.

5.2 References

- Project repository: https://github.com/sreehitha4/healthcare-appointment-patient-record-manager
- The SRS and STP supplied for this coursework.
- Current source code and automated tests in the project repository.

5.3 Tools

Python, Flask, SQLite, pytest, Git, GitHub, GitHub Actions, PlantUML or draw.io.

Team

Team member	USN
KARANAM SREEHITHA	PES1UG24AM129
KHUSHI PATEL	PES1UG24AM134
KIRTHANA S	PES1UG24AM136
PORLA NEHA	PES1UG25AM808